



Report Iniziale - Code Smile

Autore	Matricola
Daniele Pio Scaparra	NF22500101
Grazia Bonagura	NF22500105
Antonio Nappi	NF22500200

Indice

1. Descrizione del Sistema.....	2
1.1. Introduzione.....	2
1.1.1. Machine Learning Code Smells: Definizione e Tassonomia.....	2
1.1.2. Tool CodeSmile: Scopo, Obiettivi e Contesto.....	5
1.1.3. Analisi Preliminare del Sistema.....	6
2. Struttura e Architettura del Sistema.....	7
2.1. Architettura.....	7
2.2. Package del Sistema.....	7
2.2.1. cli.....	7
2.2.2. code_extractor.....	8
2.2.3. components.....	8
2.2.4. detection_rules.....	9
2.2.5. gui.....	11
2.2.6. data_preparation.....	12
2.2.7. finetuning.....	14
2.2.8. webapp.....	15
2.2.9. report.....	17
2.2.10. config.....	18
2.2.11. obj_dictionaries.....	18
2.2.12. test.....	18
2.2.13. utils.....	18
2.3. Flussi di Esecuzione.....	19

2.3.1. Flusso Static Analysis (CLI).....	19
2.3.2. Flusso GUI (Static Analysis).....	20
2.3.3. Flusso Web Application (Microservizi).....	21

1. Descrizione del Sistema

1.1. Introduzione

Lo sviluppo di sistemi software che integrano componenti di Machine Learning presenta sfide qualitative peculiari rispetto ai tradizionali sistemi software. Le pipeline di dati, la configurazione sperimentale dei modelli, la gestione della memoria GPU e l'interazione con framework specializzati (TensorFlow, PyTorch, Pandas) introducono categorie di errori e inefficienze che gli strumenti di quality assurance convenzionali non sono progettati per intercettare.

CodeSmile nasce come risposta a questa lacuna: una suite integrata di analisi statica e AI-based progettata specificamente per identificare ML-specific code smells in progetti Python. A differenza degli strumenti tradizionali di linting e code quality (pylint, flake8), CodeSmile non si limita a verificare conformità stilistiche o sintattiche, ma analizza il contesto semantico dell'uso delle librerie di Machine Learning, rilevando pattern di implementazione che, pur essendo sintatticamente corretti, compromettono prestazioni, riproducibilità o manutenibilità dei sistemi ML.

Il tool si compone di tre modalità operative complementari:

1. **Static Analysis Engine:** motore deterministico basato su regole AST (Abstract Syntax Tree) derivate dall'analisi della documentazione ufficiale delle librerie ML e da best practices consolidate;
2. **AI-Based Detection:** classificatore automatico implementato tramite fine-tuning di Large Language Models (Qwen 2.5 Coder), addestrato su dataset sintetici generati attraverso tecniche di code smell injection controllata;
3. **Web Application:** interfaccia utente moderna con architettura a microservizi, che rende accessibili entrambe le modalità di detection attraverso browser, eliminando la necessità di installazione locale.

CodeSmile è stato sviluppato nell'ambito di una ricerca empirica volta a studiare il ciclo di vita dei code smells nei sistemi ML-enabled, analizzando oltre 400000 commit provenienti da 337 progetti open source. L'obiettivo primario è fornire uno strumento pratico e scientificamente fondato per la valutazione della qualità del codice ML, supportando sviluppatori e team nella riduzione sistematica delle inefficienze implementative.

1.1.1. Machine Learning Code Smells: Definizione e Tassonomia

I Machine Learning Code Smells rappresentano una classe specializzata di antipattern implementativi che emergono dall'uso scorretto o subottimale delle API di librerie destinate al Machine Learning. A differenza dei code smells tradizionali (es. Long Method, God Class), che riguardano aspetti strutturali e di design object-oriented, gli ML code smells si manifestano nell'interazione tra codice applicativo e framework specializzati, con conseguenze dirette su:

- **Correttezza computazionale:** errori logici che alterano i risultati dell'apprendimento (es. accumulo di gradienti, inizializzazione errata);

- **Efficienza:** utilizzo inefficiente di risorse computazionali (memoria GPU, operazioni vettoriali);
- **Riproducibilità:** configurazioni non deterministiche che impediscono la replicazione degli esperimenti;
- **Manutenibilità:** codice ambiguo o fragile che complica l'evoluzione delle pipeline.

Il sistema è in grado di identificare 16 distinti code smells, organizzati in due macro categorie:

1. API-Specific Smells (6 smell)

Questa categoria raccoglie errori derivanti dall'uso improprio delle interfacce pubbliche delle librerie ML, dove metodi deprecati, sequenze di chiamata errate o scelte API subottimali generano comportamenti anomali

2. Generic Smells (10 smell)

Questa categoria include pratiche di programmazione generalmente scorrette, ma il cui impatto diventa critico nel contesto ML a causa delle dimensioni dei dati, della complessità computazionale o dei requisiti di riproducibilità.

Nella tabella seguente sono riportati tutti e 16 i code smell corredati da una breve descrizione e divisi per categoria.

Code Smell	Descrizione	Code Smell	Descrizione
Generic		Api-Specific	
Unnecessary Iteration	Uso di loop espliciti (iterrows, apply con lambda banali) su DataFrame invece di operazioni vettorizzate, degradando performance su dataset grandi.	Chain Indexing	uso di indici concatenati su DataFrame Pandas (df['col'][0]) invece di .loc[]/.iloc[], causando ambiguità copy-vs-view;
Broadcasting Feature Not Used	Uso esplicito di tf.tile() per replicare tensori invece di sfruttare il broadcasting automatico di TensorFlow, incrementando memoria e complessità.	DataFrame Conversion API Misused	utilizzo dell'attributo deprecato .values al posto del metodo esplicito .to_numpy(), con rischi di comportamento imprevedibile su ExtensionArray;
Columns and DataType Not Explicitly Set	Creazione di DataFrame o lettura da CSV senza specifica esplicita di dtype e	Gradients Not Cleared Before Backward Propagation	omissione di optimizer.zero_grad() prima di loss.backward() in PyTorch, provocando

	columns, delegando a inferenza automatica potenzialmente errata.		accumulo indesiderato di gradienti tra iterazioni;
Deterministic Algorithm Option Not Used	Attivazione permanente di <code>torch.use_deterministic_algorithms(True)</code> in produzione, sacrificando performance per determinismo non necessario post-debugging.	Matrix Multiplication API Misused	preferenza di <code>np.dot()</code> rispetto a <code>np.matmul()</code> o operatore <code>@</code> , riducendo leggibilità e coerenza con standard moderni NumPy;
Empty Column Misinitialization	Inizializzazione di colonne DataFrame con valori zero o stringhe vuote invece di <code>pd.NA/np.nan</code> , generando ambiguità semantiche.	PyTorch Call Method Misused	invocazione esplicita di <code>model.forward(x)</code> invece di <code>model(x)</code> , bypassando hook interni e meccanismi di autograd;
Hyperparameters Not Explicitly Set	Istanziamento di modelli ML (RandomForest, SVC, etc.) senza specificare iperparametri, affidandosi a default potenzialmente inadeguati e compromettendo riproducibilità.	TensorArray Not Used	uso inefficiente di <code>tf.constant()</code> in loop TensorFlow invece di <code>tf.TensorArray()</code> , causando allocazioni ripetute e degrado prestazionale.
In-Place APIs Misused	Utilizzo scorretto del parametro <code>inplace</code> in Pandas o omissione di riassegnazione su metodi non-in-place, causando <i>silent failures</i> .		
Memory Not Freed	Creazione di modelli o tensori all'interno di loop senza esplicita liberazione tramite <code>torch.cuda.empty_cache</code>		

	he(), provocando <i>memory leak</i> .		
Merge API Parameter Not Explicitly Set	Chiamate a DataFrame.merge() senza specificare how, on e validate, lasciando comportamento di join ambiguo.		
NaN Equivalence Comparison Misused	Confronti diretti con np.nan tramite operatore == invece di pd.isna()/np.isnan(), sempre valutando False per proprietà IEEE 754.		

Gli smell rilevati possono essere ulteriormente categorizzati per l'area di impatto sulla pipeline ML:

- **Data Quality:** Chain Indexing, DataFrame Conversion API Misused, Columns and DataType Not Explicitly Set, Empty Column Misinitialization, In-Place APIs Misused, Merge API Parameter Not Explicitly Set, NaN Equivalence Comparison Misused, Unnecessary Iteration (*manipolazione e trasformazione dati*);
- **Model Training:** Gradients Not Cleared Before Backward Propagation, PyTorch Call Method Misused, TensorArray Not Used, Broadcasting Feature Not Used, Hyperparameters Not Explicitly Set, Memory Not Freed (*addestramento e inferenza modelli*);
- **Reproducibility:** Deterministic Algorithm Option Not Used, Hyperparameters Not Explicitly Set, Merge API Parameter Not Explicitly Set (*configurazioni e determinismo*);
- **Performance:** TensorArray Not Used, Broadcasting Feature Not Used, Memory Not Freed, Unnecessary Iteration (*efficienza computazionale e memoria*).

La detection automatica di questi pattern consente una valutazione oggettiva e scalabile della qualità implementativa in progetti ML, superando i limiti dell'analisi manuale e fornendo metriche quantitative per il monitoraggio del debito tecnico.

1.1.2. Tool CodeSmile: Scopo, Obiettivi e Contesto

L'idea alla base del tool è che i tradizionali strumenti di software quality assurance non siano in grado di cogliere appieno le problematiche introdotte dalle complesse pipeline ML. In questo scenario, CodeSmile si propone come un AI Technical Debt Detector, capace di misurare in modo oggettivo il debito tecnico legato alle librerie di ML attraverso tre modalità di utilizzo complementari:

1. **Static Analysis Engine:** analisi basata su regole AST deterministiche;
2. **AI-Based Detection:** classificazione tramite modello linguistico fine-tuned;
3. **Web Application:** interfaccia utente moderna con architettura a microservizi.

Gli obiettivi principali del tool sono:

- Rilevamento automatico dei 16 ML-specific code smells mediante regole statiche derivate dalla letteratura e da esempi concreti;

- Supporto alla ricerca empirica, per analizzare il ciclo di vita dei code smells (introduzione, rimozione, sopravvivenza), attraverso dataset di grandi dimensioni;
- Generazione di dataset sintetici tramite injection di code smells guidata da Large Language Models (LLM), per il training di modelli di detection AI-based;
- Fine-tuning di modelli linguistici specializzati nella classificazione di ML code smells con tecniche di ottimizzazione parametrica (LoRA);
- Contributo alla qualità del software, facilitando la manutenzione e la leggibilità dei progetti che integrano componenti ML;
- Riduzione e gestione del debito tecnico legato all'intelligenza artificiale, con particolare riferimento alle seguenti categorie individuate:
 - **Data Debt:** problematiche legate alla qualità, gestione o trasformazione dei dati di input e dei DataFrame;
 - **Model Debt:** inefficienze strutturali nei modelli ML, come l'uso errato delle API di PyTorch o TensorFlow, la mancata inizializzazione corretta dei gradienti o la gestione impropria della memoria;
 - **Configuration Debt:** configurazioni non ottimali o non riproducibili, derivanti da parametri non esplicitamente dichiarati o da algoritmi non deterministici;
 - **Integrazione nel contesto SQA4AI** (Software Quality Assurance for AI), come strumento di supporto alla valutazione e alla prevenzione dei difetti legati all'uso di modelli di apprendimento automatico.

1.1.3. Analisi Preliminare del Sistema

Il funzionamento di CodeSmile si basa su tre approcci complementari di analisi:

1. Approccio AST-Based (Static Analysis)

L'analisi AST (Abstract Syntax Tree) consente di rappresentare la struttura sintattica del codice sorgente sotto forma di albero. CodeSmile utilizza questa tecnica per:

- analizzare la struttura dei file Python;
- identificare definizioni di variabili, funzioni e chiamate a metodi;
- riconoscere le dipendenze tra oggetti e librerie ML (Pandas, TensorFlow, PyTorch).

Durante la fase di parsing, il sistema costruisce un albero sintattico per ciascun file e applica un processo di estrazione semantica delle entità rilevanti: variabili, DataFrame, tensori, modelli e operazioni API. L'obiettivo è isolare le porzioni di codice potenzialmente problematiche per verificarne la conformità alle regole definite.

2. Approccio Rule-Based

A valle dell'analisi sintattica, il sistema applica regole di rilevamento (rule-based detection) derivate dalla letteratura e dalle specifiche API ML. Ogni regola corrisponde a una condizione logica che descrive il comportamento anomalo da intercettare.

Il rilevamento combina quindi analisi strutturale, per individuare pattern sintattici ricorrenti e analisi semantica per validare il contesto e ridurre i falsi positivi.

3. Approccio AI-Based

Il terzo approccio sfrutta tecniche di deep learning e transfer learning per la classificazione automatica di code smells attraverso:

- Generazione di dataset sintetici:** utilizzo di LLM (Qwen 2.5 Coder 14B) per iniettare code smells in funzioni clean, creando esempi labeled per il training;
- Fine-tuning di modelli linguistici:** adattamento di Qwen 2.5 Coder 3B tramite LoRA (Low-Rank Adaptation) su dataset di funzioni ML annotate;

- c) **Inferenza tramite API Ollama:** deployment locale del modello fine-tuned per classificazione real-time dei code smells.

Questa tripla logica consente a CodeSmile di riconoscere non solo gli errori espliciti nel codice, ma anche pratiche di programmazione inefficienti o non deterministiche, con la flessibilità di utilizzare regole deterministiche o modelli probabilistici a seconda del contesto applicativo.

2. Struttura e Architettura del Sistema

2.1. Architettura

CodeSmile adotta un'architettura modulare stratificata che integra tre paradigmi di analisi:

1. **Static Analysis Layer:** pipeline basata su traversing dell'AST e rule-based detection;
2. **AI Pipeline Layer:** generazione dati sintetici e fine-tuning di modelli linguistici;
3. **Web Application Layer:** architettura a microservizi con API Gateway e frontend React.

L'organizzazione a oggetti con separazione delle responsabilità garantisce:

- *Estensibilità:* nuove regole di detection possono essere aggiunte tramite l'interfaccia Smell;
- *Scalabilità:* esecuzione parallela su progetti multi-file e deployment distribuito dei microservizi;
- *Manutenibilità:* testing unitario, di integrazione e end-to-end su tutti i componenti;
- *Riusabilità:* componenti core (Inspector, RuleChecker) condivisi tra CLI, GUI e servizi web.

2.2. Package del Sistema

2.2.1. cli

Scopo: Interfaccia a riga di comando per avviare e configurare l'analisi statica su progetti Python.

Moduli principali: cli_runner.py: definisce la classe CodeSmileCLI. Gestisce l'interfaccia a linea di comando per l'esecuzione di CodeSmile. In particolare gestisce parametri di input/output, esecuzione sequenziale/parallela, ripresa delle analisi e multi-progetto; orchestra l'esecuzione tramite ProjectAnalyzer e salva i risultati in formato CSV. Contiene l'entrypoint main() che istanzia la CLI con parsing degli argomenti tramite argparse.

Parametri CLI supportati:

```
--input: path al progetto o directory di progetti da analizzare (obbligatorio)
--output: path di output per i risultati dell'analisi (obbligatorio)
--parallel: abilita esecuzione parallela multi-processo
--max_walkers: numero di worker per parallelizzazione (default: 5)
--resume: riprende analisi interrotta da checkpoint
--multiple: attiva modalità multi-progetto per batch analysis
```

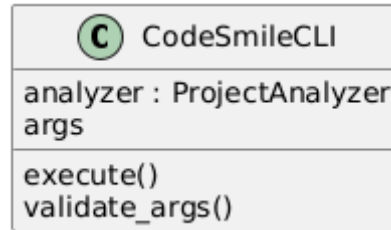


Fig 1. Diagramma che illustra CodeSmileCLI: l'interfaccia a riga di comando di CodeSmile.

2.2.2. code_extractor

Scopo: Racchiude moduli per l'estrazione delle strutture semantiche, di contesto e costrutti tipici delle librerie ML, a supporto della detection.

Moduli principali:

- **dataframe_extractor.py:** individua DataFrame Pandas, metodi richiamati e colonne accessibili via Abstract Syntax Trees (AST); usando un dizionario di metodi Pandas per affinare il rilevamento da CSV di riferimento.
- **library_extractor.py:** rileva librerie importate e alias, mappa chiamate a funzione e metodi alla libreria di riferimento tramite AST. Gestisce import statement (import X as Y) e from-import (from X import Y).
- **model_extractor.py:** gestisce le informazioni su modelli ML e operazioni su tensori. Carica dizionari di modelli/tensori da file CSV in obj_dictionaries/, filtra operazioni multi-tensore e verifica l'appartenenza di metodi a librerie note (TensorFlow, PyTorch, Keras, sklearn).
- **variable_extractor.py:** estrae definizioni e utilizzi delle variabili dal codice, collegandole ai nodi dell'AST per l'analisi statica. Traccia assegnazioni, scope e data flow.

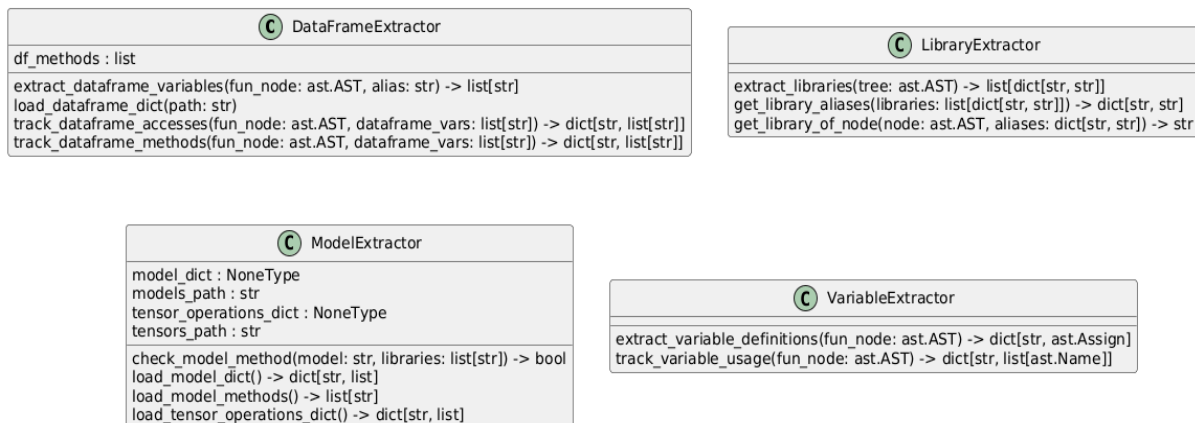


Fig 2. Diagramma che illustra i moduli principali dell'estrattore di codice (code_extractor).

2.2.3. components

Scopo: Contiene i componenti core per l'analisi dei file e dei progetti e l'applicazione delle regole di detection.

Moduli principali:

- **inspector.py:** classe Inspector che coordina l'analisi di singoli file Python. Apre ciascun file, costruisce l'AST tramite ast.parse(), coordina gli estrattori semantici (DataFrameExtractor,

LibraryExtractor, ModelExtractor, VariableExtractor) e passa i dati estratti a RuleChecker, producendo DataFrame Pandas con gli smell rilevati.

- **project_analyzer.py**: classe ProjectAnalyzer che gestisce l'analisi di interi progetti. Elabora i file tramite esecuzione sequenziale o parallela dell'analisi (con ThreadPoolExecutor), si occupa della generazione dei risultati tramite modulo report, implementa checkpoint per resume e gestisce progresso/logging.
- **project_repository_cloner.py**: classe ProjectRepositoryCloner per clonare e gestire i repositories dal dataset ML. Supporta filtri per metriche (stars, commits, LOC) e si occupa del setup e del cleanup dei progetti per l'analisi batch.
- **rule_checker.py**: classe RuleChecker che si occupa dell'applicazione delle regole di rilevamento (API-specifiche di Machine Learning e generali) sull'AST. Carica dinamicamente tutte le classi smell da detection_rules/, invoca il metodo detect() di ciascuna regola e aggrega i riscontri in DataFrame strutturato.

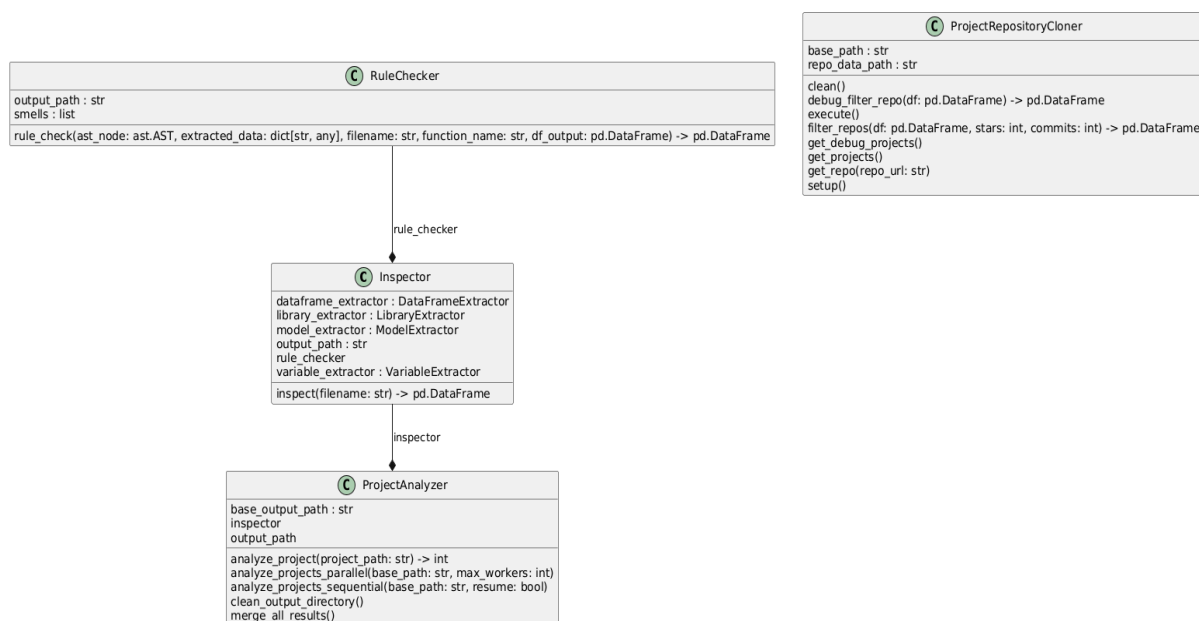


Fig 3. Diagramma che illustra i moduli principali del core analitico di CodeSmile e le loro responsabilità, inclusa l'analisi di file singoli (inspector.py), l'analisi di progetto (project_analyzer.py), la gestione del repository (project_repository_cloner.py) e l'applicazione delle regole di rilevamento degli smell (rule_checker.py).

2.2.4. detection_rules

Scopo: Contiene la gerarchia delle regole AST per individuare smell specifici di librerie AI e pattern generici, basata sulla classe astratta Smell. Sono presenti due sotto-package: api_specific_smells (Regole di rilevamento di errori nelle API di Machine Learning) e generic_smells (Regole di rilevamento per code smells comuni).

Modulo base:

- **smell.py**: superclasse astratta che definisce la struttura comune di uno smell. Ogni sottoclasse deve implementare il metodo detect(ast_node, extracted_data) che ritorna una lista di dictionary con chiavi: name, line, description, additional_info.

API-Specific Smells (sotto-package *api_specific*):

- **chain_indexing_smell.py**: rileva chained indexing su DataFrame Pandas (pattern `df['col'][0]` invece di `df.loc[0, 'col']`).
- **dataframe_conversion_api_misused.py**: segnala l'uso dell'attributo deprecato `.values` sui DataFrame Pandas, suggerendo `.to_numpy()`.
- **gradients_not_cleared_before_backward_propagation.py**: verifica che `optimizer.zero_grad()` preceda `loss.backward()` nei loop di addestramento PyTorch per evitare accumulo di gradienti.
- **matrix_multiplication_api_misused.py**: sconsiglia l'uso di `np.dot()` per moltiplicazioni matriciali dove è preferibile `np.matmul()` o l'operatore `@`.
- **pytorch_call_method_misused.py**: intercetta chiamate dirette a `forward()` nei modelli PyTorch. Suggerisce di invocare il modello direttamente tramite la sua istanza (`model(x)` invece di `model.forward(x)`).
- **tensor_array_not_used.py**: suggerisce `tf.TensorArray()` quando un `tf.constant()` viene aggiornato in loop tramite `tf.concat()`, gestendo meglio le operazioni dinamiche in TensorFlow.

Generic Smells (sotto-package *generic/*):

- **broadcasting_feature_not_used.py**: rileva uso inutile di `tf.tile()` invece del broadcasting automatico in TensorFlow per la replicazione di tensori.
- **columns_and_datatype_not_explicitly_set.py**: richiede l'esplicita impostazione di `dtype` e `columns` per DataFrame o per la lettura da CSV tramite `pd.read_csv()`.
- **deterministic_algorithm_option_not_used.py**: avvisa sull'attivazione di `torch.use_deterministic_algorithms(True)` in PyTorch per l'impatto negativo sulle prestazioni in produzione.
- **empty_column_misinitialization.py**: vieta inizializzare colonne DataFrame con 0 o stringhe vuote (`""`), consigliando `pd.NA` o `np.nan`.
- **hyperparameters_not_explicitly_set.py**: segnala modelli ML istanziati senza specificare esplicitamente gli iperparametri (es. `RandomForestClassifier()` senza parametri).
- **in_place_apis_misused.py**: controlla l'uso coerente del parametro `inplace` e l'assegnazione del risultato nelle API Pandas, segnalando quando viene omesso o impostato a `False` senza riassegnazione.
- **memory_not_freed.py**: impone l'uso di `del` e `torch.cuda.empty_cache()` quando si creano modelli in loop, per liberare la memoria esplicitamente ed evitare memory leak.
- **merge_api_parameter_not_explicitly_set.py**: richiede parametri espliciti `how`, `on` e opzionalmente `validate` nelle chiamate a `merge()` di Pandas per evitare comportamenti ambigui.
- **nan_equivalence_comparison_misused.py**: evita confronti diretti con `np.nan` (es. `x == np.nan`), suggerendo `pd.isna()` o `np.isnan()`.
- **unnecessary_iteration.py**: rileva loop/operazioni inefficienti su DataFrame (`iterrows`, `apply`, `applycon`, `lambda` semplici, `applymap`) dove operazioni vettorizzate sarebbero più performanti.

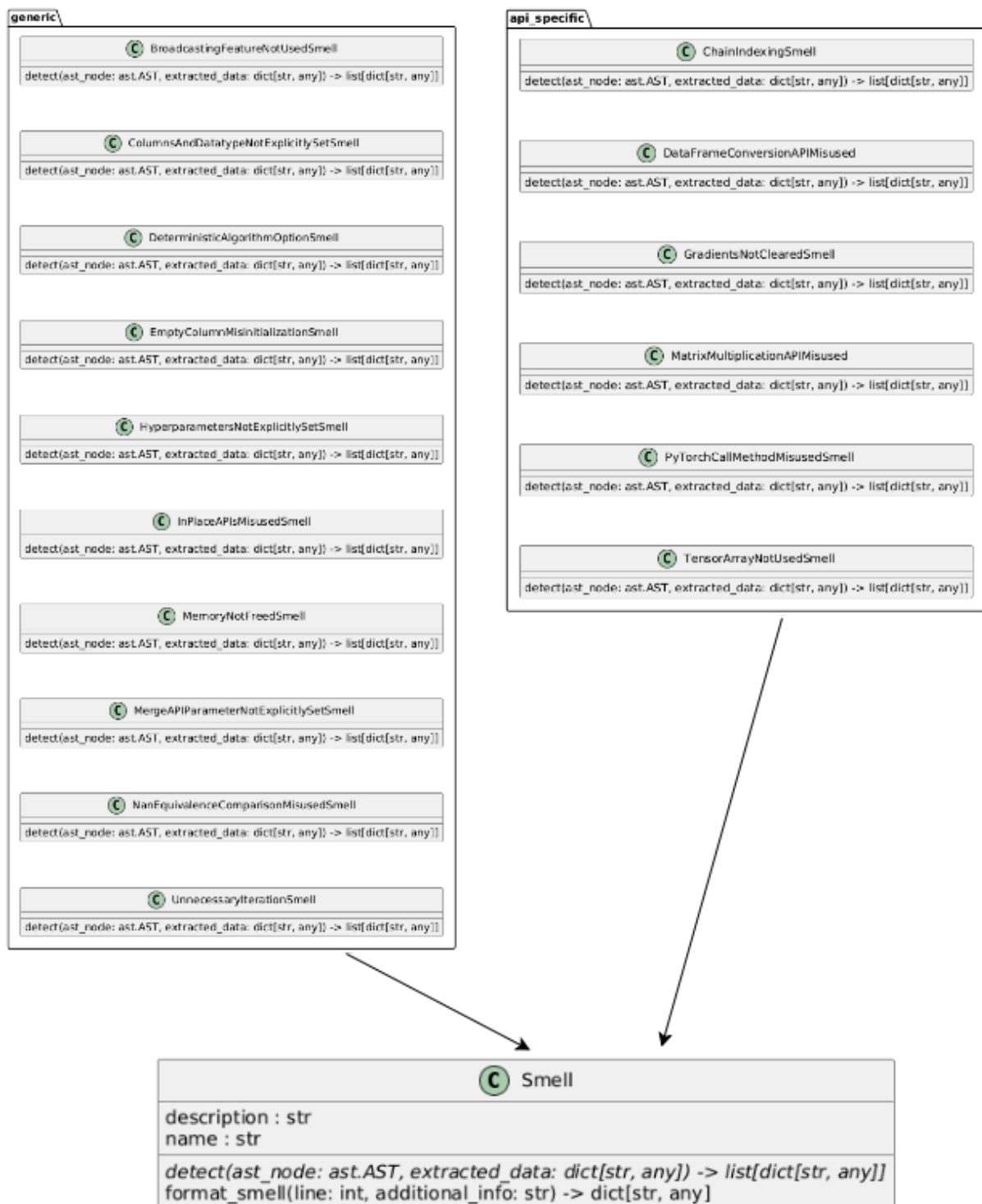


Fig 4. Struttura e contenuti del package `detection_rules` di CodeSmile, che include regole specifiche per API AI e regole generiche per i code smells comuni.

2.2.5. gui

Scopo: Interfaccia grafica basata su Tkinter per configurare ed eseguire l'analisi con visualizzazione dei log in tempo reale.

Moduli principali:

- **code_smell_detector_gui.py**: classe CodeSmellDetectorGUI che costruisce l'interfaccia grafica per l'analisi dei code smells. Permette la raccolta di percorsi input/output e parametri (numero walker, flags parallel/resume/multiple), e lancia l'analisi su thread dedicato tramite ProjectAnalyzer per non bloccare l'UI. Include gestione di bottoni, spinner e checkbox Tkinter.
- **gui_runner.py**: script entry point che crea l'istanza Tk() e avvia il mainloop tramite l'utilizzo di CodeSmellDetectorGUI.
- **textbox_redirect.py**: implementa un writer io.StringIO che redirige stdout nel widget Text della GUI. Consente la visualizzazione realtime dei diversi log generati dal tool durante l'analisi.

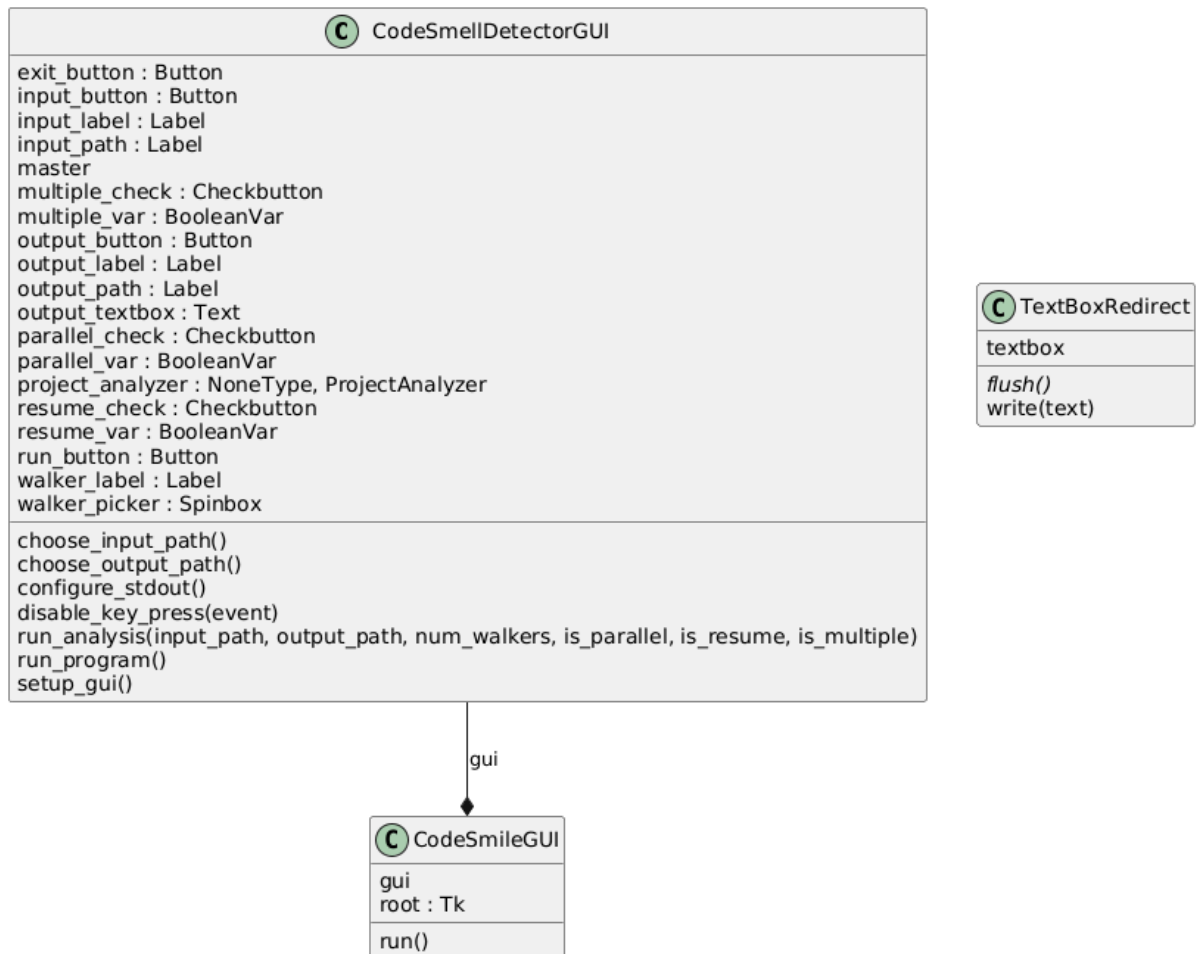


Fig 5. Descrizione del modulo gui e dei suoi componenti per l'interfaccia grafica utente del tool.

2.2.6. data_preparation

Scopo: Pipeline completa per la generazione di dataset sintetici di training tramite injection di ML code smells guidata da Large Language Models.

Moduli principali:

- **dataset_creation_runner.py**: classe DatasetCreationRunner che orchestra l'intera pipeline di creazione dataset in 4 step sequenziali:
 - Step 1: clonazione repository GitHub con progetti ML;
 - Step 2: estrazione funzioni ML-related tramite FunctionDatasetBuilder;
 - Step 3: analisi statica delle funzioni con CodeSmellAnalyzer;

- Step 4: injection di smell sintetici tramite CodeSmellInjector.
- **function_dataset_builder.py**: classe FunctionDatasetBuilder che estrae funzioni Python da repository clonati. Utilizza AST per:
 - filtrare file ML-related basandosi su import di librerie (pandas, numpy, torch, tensorflow, sklearn);
 - identificare funzioni che contengono keyword ML (fit, predict, transform, model, loss, optimizer, etc.);
 - estrarre codice sorgente delle funzioni con metadati (nome, file path). Supporta esecuzione parallela con ThreadPoolExecutor e salvataggio in formato JSON.
- **code_smell_analyzer.py**: classe CodeSmellAnalyzer che applica il motore di analisi statica (Inspector + RuleChecker) su dataset di funzioni estratte. Analizza in parallelo con max_workers configurabile e salva risultati annotati (funzione + smell rilevati) per filtrare funzioni clean.
- **code_smell_injector.py**: classe CodeSmellInjector che inietta artificialmente code smells in funzioni clean usando LLM. Caratteristiche:
 - Dizionario di 16 smell con descrizioni dettagliate, esempi e rationale;
 - Prompt engineering per guidare l'LLM nell'injection controllata;
 - Selezione casuale di 1-N smell per funzione (configurabile);
 - Validazione sintattica del codice generato. Supporta interfaccia BaseLLM per diversi modelli (Qwen via Ollama).
- **injected_smells_dataset_builder.py**: classe InjectedSmellsDatasetBuilder che costruisce il dataset finale di training. Struttura JSON con:
 - function_data: codice funzione (clean o smelly);
 - smelly: boolean flag;
 - smells: lista di smell con nome, descrizione e info aggiuntive. Bilancia dataset tra funzioni clean e smelly per evitare class imbalance.
- **dataset_evaluator.py**: classe DatasetEvaluator per validazione qualitativa del dataset. Calcola metriche di distribuzione smell, analizza lunghezza funzioni e verifica consistenza delle annotazioni.

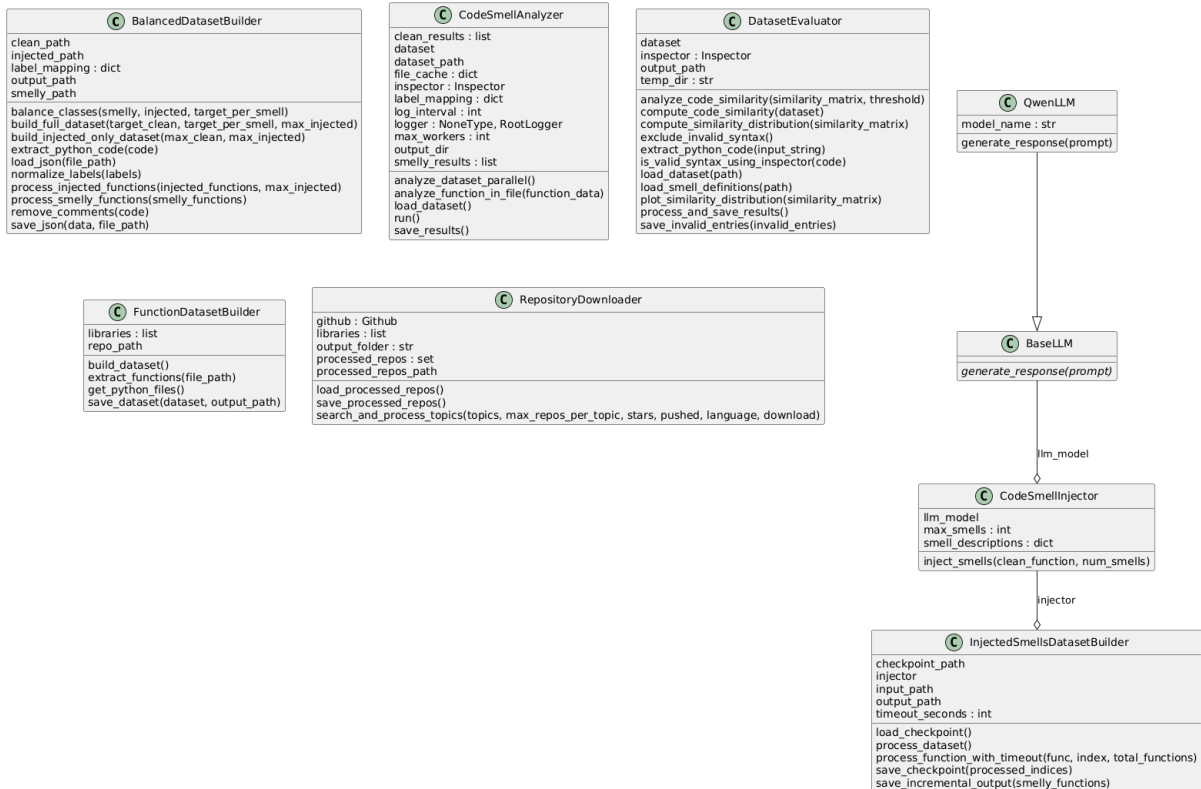


Fig 6. Architettura della pipeline data_preparation per la generazione di dataset sintetici di training per l'identificazione di ML code smells, che orchestra la clonazione di repository, l'estrazione di funzioni, l'analisi statica e l'iniezione guidata da LLM di smell artificiali.

2.2.7. finetuning

Scopo: Pipeline di fine-tuning e validazione di modelli linguistici specializzati nella detection di ML code smells tramite tecniche di Parameter-Efficient Fine-Tuning (PEFT).

Sotto-package *train*:

- **model_trainer.py**: classe ModelTrainer che gestisce:
 - Caricamento di modelli preaddestrati via FastLanguageModel (libreria unsloth);
 - Applicazione di LoRA (Low-Rank Adaptation) per fine-tuning efficiente con parametri configurabili: rank r, target_modules, lora_alpha, lora_dropout;
 - Applicazione di chat template (Qwen-2.5 format) al tokenizer per formattazione conversazionale;
 - Supporto per 4-bit quantization tramite load_in_4bit per riduzione memoria.
- **data_loader.py**: classe DataLoader che prepara dataset per training:
 - Caricamento da JSON con struttura {function_data, smelly, smells};
 - Split train/validation con sklearn.model_selection.train_test_split;
 - Tokenizzazione con chat
 - Gestione di etichette multi-smell (lista) e conversione a formato HuggingFace Dataset.
- **train_runner.py**: script entry point per avviare il training. Configura:
 - TrainingArguments di HuggingFace (batch size, learning rate, epochs, gradient accumulation);
 - SFTTrainer (Supervised Fine-Tuning) con packing, max sequence length;
 - Salvataggio checkpoint e modello finale in formato LoRA adapter.

Sotto-package *validation*:

- **model_inference.py**: classe `ModelInference` per inferenza su modello fine-tuned:
 - Caricamento modello da checkpoint LoRA;
 - Applicazione `FastLanguageModel.for_inference()` per ottimizzazione;
 - Generazione con parametri configurabili: `max_new_tokens`, `temperature`, `min_p`;
 - Supporto per device selection (CPU/CUDA).

Modello utilizzato:

- Base: Qwen 2.5 Coder 3B
- Tecnica: LoRA con rank 16-32
- Quantizzazione: 4-bit via `bitsandbytes`
- Performance: 94.46% accuracy, ~95% precision/recall/F1

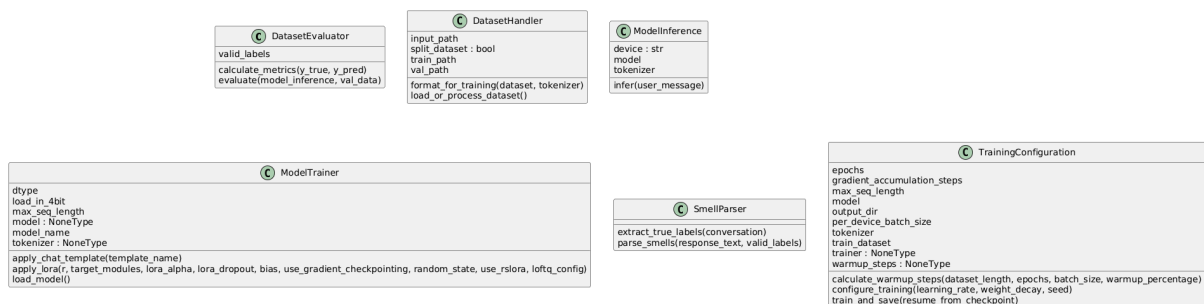


Fig 7. Diagramma della Pipeline di fine-tuning e validazione del modello Qwen 2.5 Coder 3B specializzato nella *detection* di *ML code smells*, utilizzando tecniche di PEFT (LoRA) e quantizzazione a 4-bit per ottimizzare l'efficienza. La pipeline include moduli dedicati al caricamento dati (*data_loader*), addestramento (*model_trainer*, *train_runner*) e inferenza (*model_inference*).

2.2.8. webapp

Scopo: Applicazione web moderna con architettura a microservizi per analisi interattiva di code smells tramite interfaccia browser.

Architettura: API Gateway + 3 servizi backend (FastAPI) + 1 frontend (Next.js/React), orchestrati tramite Docker Compose.

Componente Gateway:

- **gateway/main.py**: API Gateway FastAPI (porta 8000) che funge da reverse proxy per:
 - `/api/detect_smell_ai` → forward a AI Analysis Service (porta 8001);
 - `/api/detect_smell_static` → forward a Static Analysis Service (porta 8002);
 - `/api/generate_report` → forward a Report Service (porta 8003). Implementa CORS middleware per comunicazione con frontend e utilizza `httpx.AsyncClient` per chiamate asincrone ai servizi.

Servizio AI Analysis:

- **services/aiservice/app/main.py**: FastAPI application per detection AI-based. Registra router `/detect_smell_ai` e configura CORS.
- **services/aiservice/app/routers/detect_smell.py**: endpoint POST che:
 - Riceve payload `{code_snippet: str}`;
 - Invoca `Model.detect_code_smell()`;
 - Ritorna `{success: bool, smells: List[Smell]}`.
- **services/aiservice/app/utils/model.py**: classe `Model` che interagisce con Ollama API:

- Endpoint: `http://localhost:11434/api/generate`;
- Modello: `codesmile: latest` (Qwen fine-tuned convertito in `.gguf`);
- Streaming response parsing con accumulo chunk JSON;
- Metodo `parse_smell()` per estrazione strutturata degli smell dal testo LLM.
- **services/aiservice/app/schemas/**: definisce Pydantic models per request/response validation:
 - `DetectSmellRequest`: schema input con `code_snippet`;
 - `DetectSmellResponse`: schema output con success e lista Smell;
 - `Smell`: modello singolo smell con `smell_name`, `description`, `line`, `additional_info`.

Servizio Static Analysis:

- **services/staticanalysis/app/main.py**: FastAPI application per detection rule-based. Registra router `/detect_smell_static`.
- **services/staticanalysis/app/routers/detect_smell.py**: endpoint POST che applica analisi statica.
- **services/staticanalysis/app/utils/static_analysis.py**: funzione `detect_static()` che:
 - Crea temporary file con codice snippet;
 - Invoca Inspector del core engine;
 - Converte DataFrame risultati in lista Smell;
 - Cleanup del temporary file.

Servizio Report:

- **services/report/app/main.py**: FastAPI application per generazione report aggregati.
- **services/report/app/routers/report.py**: endpoint POST `/generate_report` che:
 - Riceve lista di progetti analizzati con smell;
 - Genera report Excel/CSV con statistiche aggregate;
 - Ritorna file o summary JSON.

Frontend Next.js:

- **app/page.tsx**: homepage con card navigation verso sezioni:
 - Analyze Python Code (singolo file);
 - Analyze Project (folder upload);
 - Generate Reports;
 - About.
- **app/upload-python/page.tsx**: pagina per upload e analisi singolo file:
 - Toggle AI/Static mode selection;
 - File input con validazione `.py`;
 - Progress bar animata con Framer Motion;
 - Display risultati con card per ogni smell rilevato;
 - Gestione errori e toast notifications.
- **app/upload-project/page.tsx**: pagina per analisi progetto multi-file:
 - Supporto per multiple file upload;
 - Selezione mode (AI/Static) per ciascun file;
 - Aggregazione risultati in tabella interattiva.
- **components/**: componenti React riutilizzabili:
 - `HeaderComponent.tsx`: navbar con logo e navigation;
 - `FooterComponent.tsx`: footer con link e copyright;
 - Altri componenti UI (buttons, cards, modals).
- **utils/api.ts**: client TypeScript per chiamate API:
 - `detectAi(codeSnippet)`: POST a `/api/detect_smell_ai`;
 - `detectStatic(codeSnippet)`: POST a `/api/detect_smell_static`;
 - `generateReport(projects)`: POST a `/api/generate_report`;

- Error handling centralizzato con Axios interceptors.

Deployment:

- docker-compose.yml: orchestrazione di 5 container:
 - gateway: porta 8000, dipende da servizi backend;
 - ai_analysis_service: porta 8001, variabile env OLLAMA_API_URL;
 - static_analysis_service: porta 8002, monta volume con codice core;
 - report_service: porta 8003;
 - webapp: porta 3000, variabile env NEXT_PUBLIC_API_BASE_URL. Networking con bridge network per comunicazione interna.

Testing webapp:

- cypress/: test E2E con Cypress:
 - upload-python. cy.ts: test workflow upload singolo file;
 - upload-project.cy.ts: test workflow upload progetto;
 - Fixtures con sample code smells.
- tests/: test unit Jest per componenti React:
 - Rendering tests;
 - Interaction tests con React Testing Library;
 - Mock API calls.

2.2.9. report

Scopo: Generazione di report aggregati e visualizzazioni sugli smell rilevati.

Modulo principale: report_generator.py: classe ReportGenerator con metodi:

- _find_project_details(): localizza folder project_details/ con CSV risultati;
- _load_data(): carica e merge multipli CSV in DataFrame Pandas;
- smell_report(): genera CSV con conteggio occorrenze per tipo smell;
- project_report(): genera CSV con totale smell per progetto;
- summary_report(): genera Excel multi-sheet con overview generale + dettaglio per progetto;
- visualize_smell_report(): crea bar chart con matplotlib e salva PNG;
- menu(): interfaccia CLI interattiva per selezione tipo report;
- run(): orchestratore principale con gestione input/output paths.

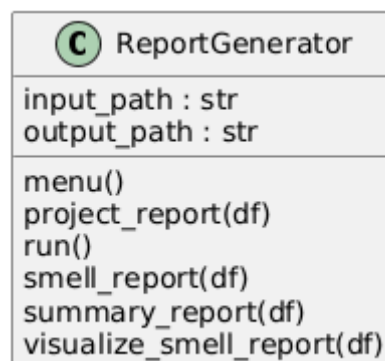


Fig 8. Questo modulo Python (report_generator.py) è il cuore della generazione di report e visualizzazioni sugli "smell" di codice rilevati, offrendo una suite di metodi per l'aggregazione, l'analisi e la presentazione dei dati.

2.2.10. config

Scopo: File di configurazione YAML per parametrizzazione regole e threshold.

File principali:

- **smell_thresholds.yaml:** threshold numerici per detection (es. max lunghezza funzione, max complessità ciclomatica).
- **library_mappings.yaml:** mappature tra alias librerie e nomi canonici.

2.2.11. obj_dictionaries

Scopo: Dizionari CSV di supporto per l'estrazione semantica.

File principali:

- **models.csv:** lista modelli ML con colonne id, library, method. Include modelli da TensorFlow (Model, Sequential), sklearn (classifiers, regressors), PyTorch, Keras.
- **tensor_operations.csv:** operazioni tensoriali con colonne id, library, operation, number_of_tensors_input. Filtrate per operazioni multi-tensore.
- **dataframe_methods.csv:** metodi Pandas DataFrame/Series per riconoscimento operazioni.

2.2.12. test

Scopo: Suite completa di testing per assicurare qualità e correttezza del sistema.

Struttura:

- **test/unit_testing/:** test unitari per singoli componenti:
 - detection_rules/api_specific/: test per ogni smell API-specific con casi positivi e negativi;
 - detection_rules/generic/: test per ogni smell generic;
 - components/: test per Inspector, RuleChecker, ProjectAnalyzer;
 - code_extractor/: test per estrattori semantici;
 - gui/: test per componenti GUI con mock Tkinter.
- **test/integration_testing/:** test di integrazione:
 - Pipeline completa CLI su progetto sample;
 - Coordinamento Inspector + RuleChecker + Extractors;
 - Workflow data_preparation.
- **test/system_testing/:** test end-to-end:
 - Esecuzione completa su progetti reali;
 - Validazione output CSV/Excel;
 - Performance testing su dataset grandi.

Framework utilizzati:

- pytest per test Python con fixtures e parametrizzazione;
- pytest-cov per code coverage;
- unittest.mock per mocking di componenti esterni;
- Cypress per E2E testing webapp;
- Jest + React Testing Library per unit testing frontend.

Configurazione:

- .coveragerc: configurazione coverage con esclusione di test/ e GUI.
- pytest.ini (implicito): configurazione pytest con markers e plugins.

2.2.13. utils

Scopo: Utility functions trasversali e client API per la comunicazione tra frontend e backend services.

Moduli principali:

- **api.ts:** client HTTP TypeScript per l'interazione con i microservizi backend. Implementa:

- Configurazione Axios:
 - axiosInstance con baseURL da environment variable (NEXT_PUBLIC_API_BASE_URL) o default http://localhost:8000/api;
 - Timeout di 500 secondi per analisi su progetti di grandi dimensioni.
- Error Handling Centralizzato:
 - handleErrorResponse(): gestione uniforme errori con logging e fallback data type-safe.
- API Functions:
 - detectAi(codeSnippet: string): POST a /detect_smell_ai per analisi AI-based. Ritorna {success: bool, smells: array}.
 - detectStatic(codeSnippet: string): POST a /detect_smell_static per analisi statica. Ritorna {success: bool, smells: array}.
 - generateReport(projects: any[]): POST a /generate_report per generazione report aggregati. Ritorna {report_data, message}.

Type Safety: import di DetectResponse e GenerateReportResponse da @/types/types con validazione runtime tramite type guards.

2.3. Flussi di Esecuzione

CodeSmile supporta tre modalità operative distinte, ciascuna progettata per specifici contesti d'uso. Di seguito vengono dettagliati i flussi di esecuzione con relativi diagrammi di sequenza.

2.3.1. Flusso Static Analysis (CLI)

Il flusso CLI è progettato per analisi batch automatizzate, integrazione in pipeline CI/CD e processing di grandi repository.

Sequenza di esecuzione:

1. Inizializzazione: parsing argomenti CLI e validazione parametri input/output
2. Discovery: scansione ricorsiva dei file Python nel path di input
3. Analisi parallela/sequenziale: per ogni file:
 - Parsing AST con ast.parse()
 - Estrazione semantica (librerie, variabili, DataFrame, modelli)
 - Applicazione regole detection tramite RuleChecker
4. Aggregazione risultati: costruzione DataFrame con smell rilevati
5. Persistenza: salvataggio CSV nella sottodirectory project_details/ creata dinamicamente all'interno dell'output path specificato dall'utente
6. Report opzionale: generazione summary Excel/visualizzazioni

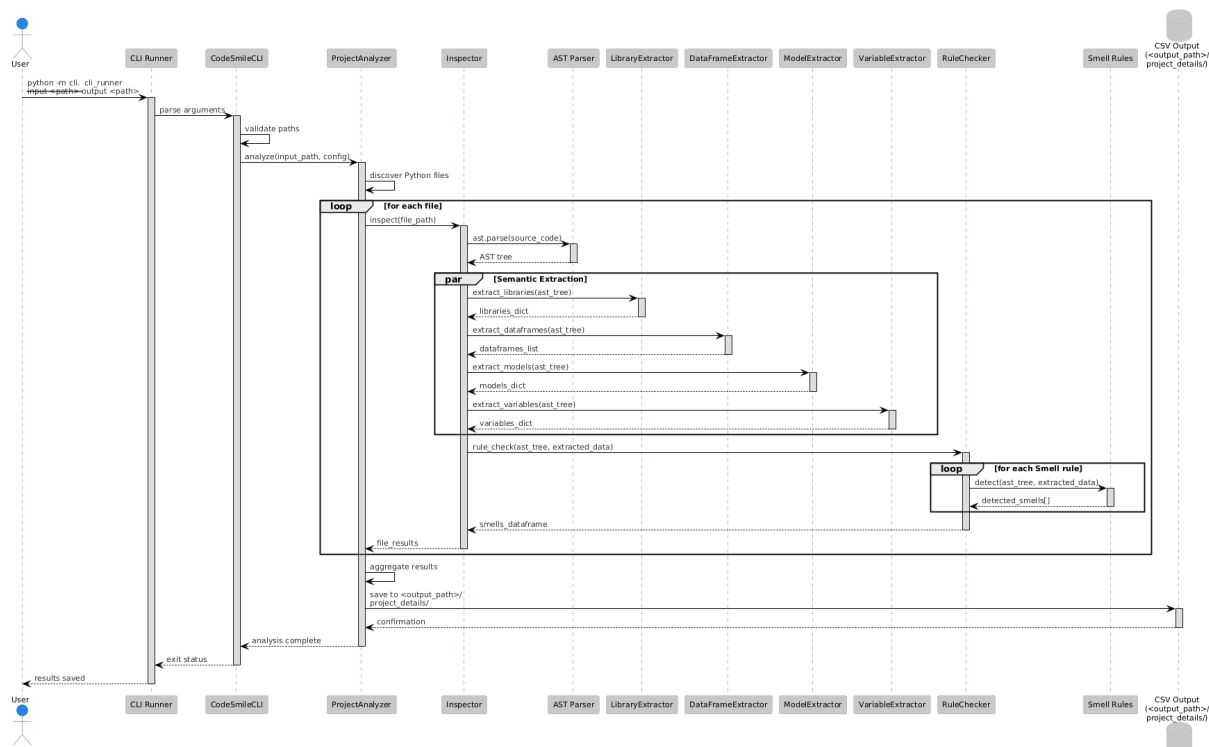


Fig 9. Sequence Diagram del Flusso CLI

2.3.2. Flusso GUI (Static Analysis)

Il flusso GUI fornisce un'interfaccia grafica Tkinter per utenti non tecnici, con visualizzazione real-time dei log e configurazione interattiva.

Sequenza di esecuzione:

1. Inizializzazione GUI: rendering finestra Tkinter con widgets
2. Configurazione utente: selezione paths, numero walker, flags (parallel/resume/multiple)
3. Esecuzione thread dedicato: analisi su thread separato per non bloccare UI
4. Redirezione output: stdout → Text widget per logging real-time
5. Analisi: identica a flusso CLI tramite ProjectAnalyzer
6. Feedback utente: messaggi di completamento e gestione errori

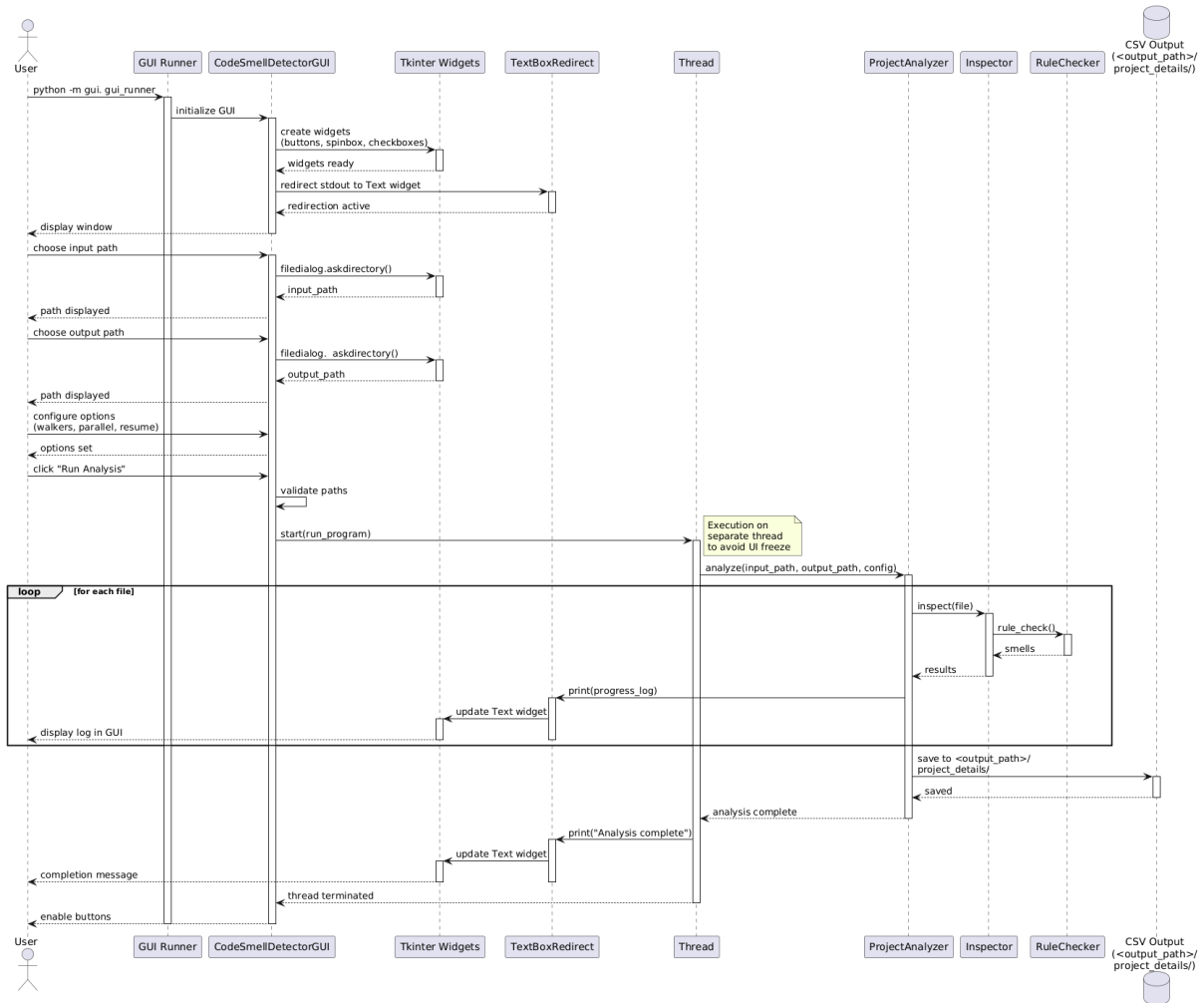


Fig 10. Sequence Diagram del Flusso GUI

2.3.3. Flusso Web Application (Microservizi)

Il flusso webapp implementa un'architettura a microservizi con API Gateway, supportando sia analisi AI-based che static analysis attraverso interfaccia browser moderna.

Sequenza di esecuzione:

1. Frontend rendering: Next.js SSR/CSR della pagina upload
2. User interaction: upload codice + selezione mode (AI/Static)
3. API call: POST al Gateway via axios client
4. Routing Gateway: forward alla service appropriato (AI/Static)
5. Processing backend:
 - AI mode: chiamata Ollama API → parsing response LLM
 - Static mode: creazione temp file → Inspector → parsing results
6. Response chain: service → Gateway → Frontend
7. UI update: rendering smell cards con animazioni

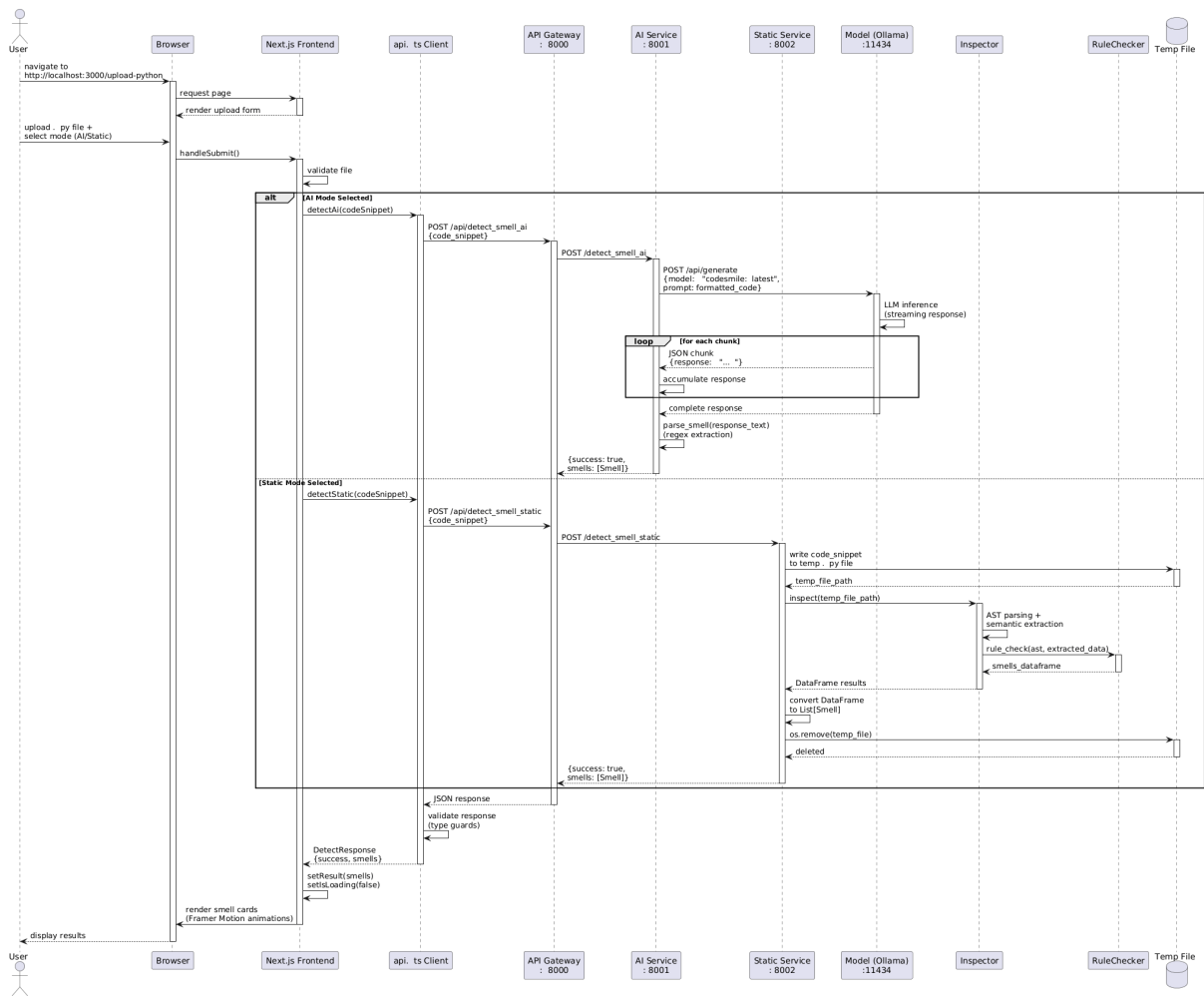


Fig 11. Sequence Diagram del Flusso WebApp